S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 11
Postlab (OPEN ENDED PRACTICAL)

Aim: To implement and analyze different Page Replacement Algorithms (FIFO and LRU) and compare their performance based on page faults.

Name: Trisha Prakash Pillewan

USN: CM24055

Semester / Year:4th/2nd

Academic Session:2025-26

Date of Performance:

Date of Submission:

❖ **Aim:** To implement and analyze different Page Replacement Algorithms (FIFO and LRU) and compare their performance based on page faults.

❖ **Objectives:**

- To understand page replacement algorithms.
- To calculate number of page faults.
- To compare FIFO and LRU performance.

❖ **Requirements:**

✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Compiler: GCC
- Text Editor: Nano, Vim or any editor

❖ **Theory:**

Page Replacement is a memory management technique used by the operating system when the main memory is full and a new page needs to be loaded. In such cases, the OS replaces an existing page with a new one based on a specific algorithm.
It is an important concept in virtual memory systems, where only a limited number of pages can be kept in memory at a time. The efficiency of a page replacement algorithm is measured by the number of **page faults** it generates.
A **page fault** occurs when a page is not found in memory and needs to be loaded from disk.
Efficient page replacement algorithms help in reducing page faults and improving system performance.

❖ **Types of Algorithms Used:**

1. FIFO (First In First Out)

FIFO replaces the page that has been in memory for the longest time.

- Simple to implement
- Uses queue structure
- May lead to more page faults
- Suffers from Belady's anomaly

2. LRU (Least Recently Used)

LRU replaces the page that has not been used for the longest period of time.

- More efficient than FIFO
- Reduces page faults
- Uses past reference history

- Easy to understand and implement
- No starvation (every process gets a chance)
- However, it may lead to **high waiting time**
- Suffers from **convoy effect**, where short processes wait for long processes

2. SJF (Shortest Job First)

SJF selects the process with the **smallest burst time** for execution. It is more efficient than FCFS in terms of minimizing waiting time.

- Provides **minimum average waiting time**
- More efficient and faster compared to FCFS
- Difficult to predict exact burst time in real scenarios
- May cause **starvation** of longer processes
- Slightly complex to implement

❖ **Important Terms**

1. Page Fault

A page fault occurs when a required page is not present in memory.

👉 Lower page faults = Better performance

2. Frame

A frame is a fixed-size block of main memory used to store pages.

❖ **CODE:**

```
#include <stdio.h>

// FIFO Algorithm
void fifo(int pages[], int n, int capacity) {
    int frame[10], count = 0, index = 0;

    for(int i = 0; i < capacity; i++)
        frame[i] = -1;

    for(int i = 0; i < n; i++) {
        int found = 0;

        for(int j = 0; j < capacity; j++) {
            if(frame[j] == pages[i]) {
                found = 1;
                break;
            }
        }

        if(!found) {
            frame[index] = pages[i];
            index = (index + 1) % capacity;
            count++;
        }
    }

    printf("\nFIFO Page Faults = %d\n", count);
}

// LRU Algorithm
void lru(int pages[], int n, int capacity) {
    int frame[10], time[10], count = 0, t = 0;

    for(int i = 0; i < capacity; i++) {
        frame[i] = -1;
        time[i] = 0;
    }

    for(int i = 0; i < n; i++) {
        int found = 0;

        for(int j = 0; j < capacity; j++) {
            if(frame[j] == pages[i]) {
                t++;
                time[j] = t;
                found = 1;
```

```
            break;
        }
    }

    if(!found) {
        int min = 0;
        for(int j = 1; j < capacity; j++) {
            if(time[j] < time[min])
                min = j;
        }

        frame[min] = pages[i];
        t++;
        time[min] = t;
        count++;
    }
  }

  printf("LRU Page Faults = %d\n", count);
}

int main() {
  int pages[20], n, capacity;

  printf("Enter number of pages: ");
  scanf("%d", &n);

  printf("Enter page reference string:\n");
  for(int i = 0; i < n; i++)
    scanf("%d", &pages[i]);

  printf("Enter number of frames: ");
  scanf("%d", &capacity);

  fifo(pages, n, capacity);
  lru(pages, n, capacity);

  return 0;
   }
```

❖ **Output:**

```
Output
Enter number of pages: 7
Enter page reference string:
1 3 0 3 5 6 3
Enter number of frames: 3

FIFO Page Faults = 6
LRU Page Faults = 5


=== Code Execution Successful ===
```

❖ Conclusion:

Page replacement algorithms are important for efficient memory management. In this practical, FIFO and LRU algorithms were implemented and compared.
It was observed that:

- FIFO is simple but may result in more page faults.
- LRU is more efficient as it reduces page faults by using recent usage history.

Thus, LRU performs better than FIFO in most cases, but it is more complex to implement.

❖ Discussion Questions:

1. **What is a page fault?**
2. **What is the difference between FIFO and LRU?**
3. **What is Belady's anomaly?**
4. **Why is LRU better than FIFO?**
5. **What is a frame in memory management?**

❖ References:

- **https://www.geeksforgeeks.org/page-replacement-algorithms/**
- **https://chatgpt.com/**

Date:____/____/2026

Signature Course Coordinator B.Tech CSE(AIML)
Sem: 4 / 2025-26